

Candidate Spec: Balanced Ternary Graph Token Grammar

Balanced Ternary Graph Token Grammar - Candidate Specification

This page documents the **candidate version** of the balanced-ternary graph token grammar for mathematical reasoning. It is intended for testing, refinement, and eventual integration into the Lean DSL $\rightarrow$ IR $\rightarrow$ SLM pipeline.

The grammar defines a three-axis token structure:

- **Axis 1 - Class Prefix:** E, R, O, A
- **Axis 2 - Polarity:** +, 0, -
- **Axis 3 - Chain Marker:** *

This structure ensures that **the next token corresponds to the next mathematical step**, enabling transformer-friendly reasoning.

1. Lexical Categories

Token Classes

- **Entities:** variables, numbers, terms, equations
- **Relations:** structural edges (mul, add, sub, div, eq)
- **Operations:** rewrite steps (add, sub, div, iso)
- **Attributes:** metadata (value, type)

Prefixes

- E - entity
- R - relation
- O - operation
- A - attribute

Polarity

- + - constructive
- 0 - neutral
- - - reductive

Chain Marker

- * - required on all entity references

2. Token Forms (BNF)

```
<entity-token> ::= "E" <polarity> ":" <entity-id> "*"

<relation-token> ::= "R" <polarity> ":" <relation-op> "*"
                  ":" <entity-ref> ":" <entity-ref> ":" <entity-ref>

<operation-token> ::= "O" <polarity> ":" <operation-op> "*"
                   ":" <entity-ref> ":" <entity-ref>

<attribute-token> ::= "A" <polarity> ":" <entity-id> "*"
                   ":" <attr-key> ":" <attr-value>

<polarity> ::= "+" | "0" | "-"

<relation-op> ::= "mul" | "add" | "sub" | "div" | "eq"
<operation-op> ::= "add" | "sub" | "div" | "iso"

<entity-id> ::= "X" | "Y" | "T1" | "T2" | "EQ" | "N" <int> | <custom-id>
<entity-ref> ::= "E" <polarity> ":" <entity-id> "*"

<attr-key> ::= "val" | "type" | <custom-key>
<attr-value> ::= <int> | <string>
```

3. Phrase-Level Grammar

```
<equation-graph> ::= <entity-block> <attribute-block> <relation-block> <operation-block>

<entity-block> ::= { <entity-token> }
<attribute-block> ::= { <attribute-token> }
<relation-block> ::= { <relation-token> }
<operation-block> ::= { <operation-token> }
```

Semantic Constraints

- Every entity reference must correspond to a declared entity.
- Polarity must match operation type:
 - $R+ \rightarrow \text{mul, add}$

- R- → sub, div
- R0 → eq
- O+ → add
- O- → sub, div
- O0 → iso
- Chain marker * is mandatory.

4. Example Encoding

Equation: $3x + 5 = 12$

```
E0:X*
E0:N3*
E0:N5*
E0:N12*
E0:T1*
E0:T2*
E0:EQ*

A0:N3*:val:3
A0:N5*:val:5
A0:N12*:val:12
A0:X*:type:unknown

R+:mul*:E0:X*:E0:N3*:E0:T1*
R+:add*:E0:T1*:E0:N5*:E0:T2*
R0:eq*:E0:T2*:E0:N12*:E0:EQ*

O-:sub*:E0:N5*:E0:EQ*
O-:div*:E0:N3*:E0:EQ*
O0:iso*:E0:X*:E0:EQ*
```

5. Purpose of This Grammar

This grammar is the **candidate version** for testing:

- transformer-friendly tokenization
- polarity-aligned next-step prediction
- reversible mapping to Lean DSL
- compatibility with the SLM rewrite engine

It will evolve as we test:

- entropy reduction
- prediction locality
- Lean integration
- IR compression ratio

This page serves as the canonical reference for the current version.